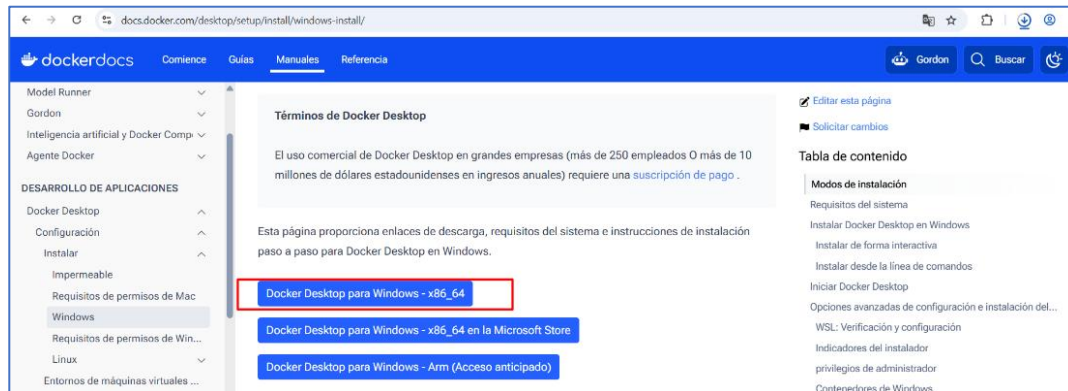
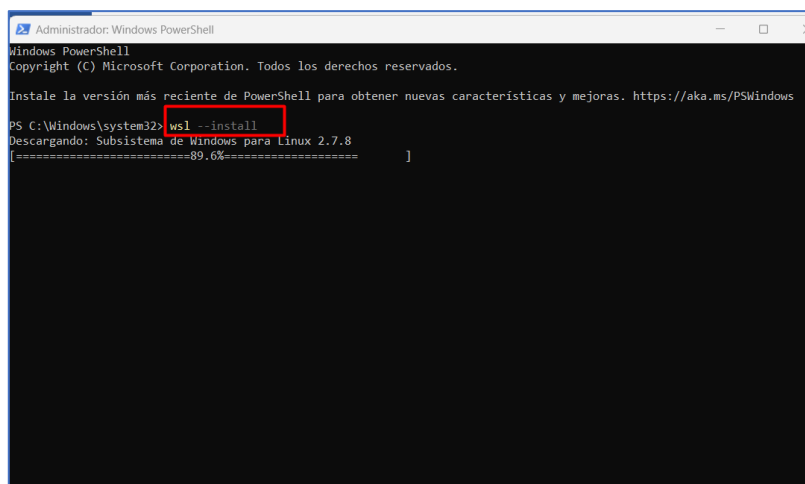


EVIDENCIAS DEL PROCESO: INSTALACIÓN, CONFIGURACIÓN Y VALIDACIÓN

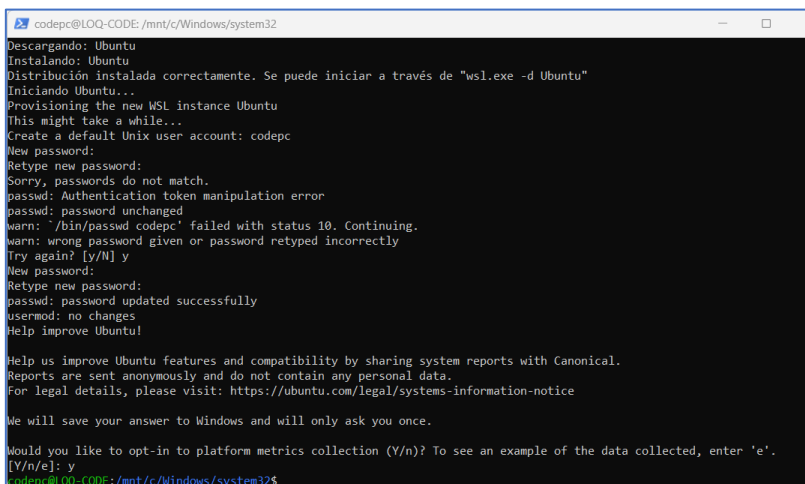
Descarga de Docker Desktop: <https://docs.docker.com/desktop/setup/install/windows-install/>

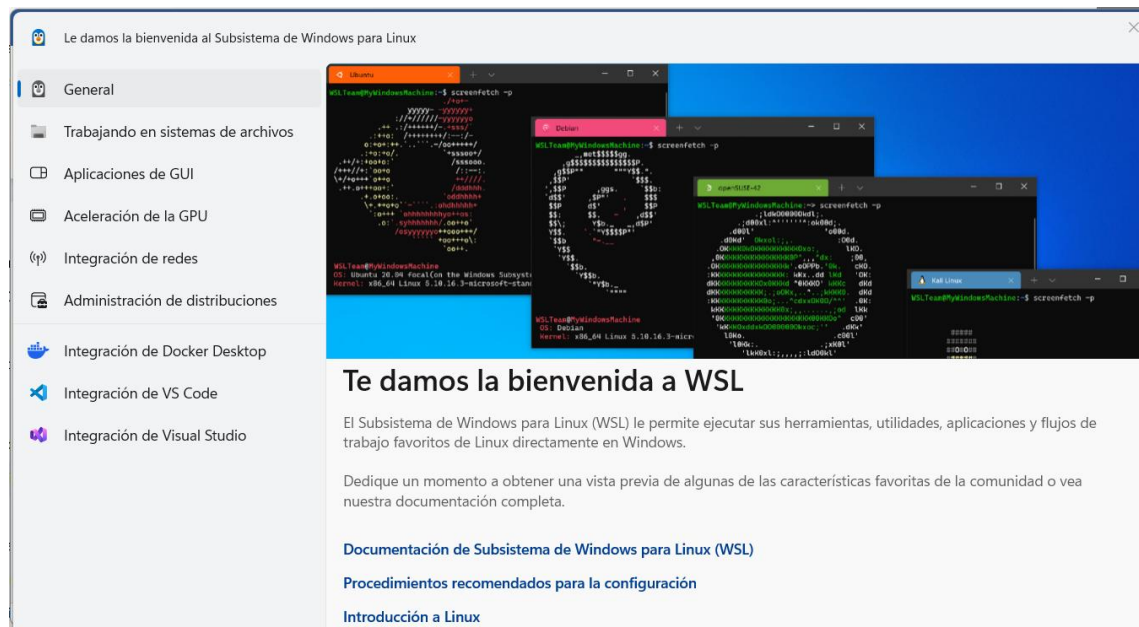


Descargando WSL en Windows:



Finalización WSL en Windows:





Creación Cuenta DataBricks:


Free Edition

Set up your account

Name your account

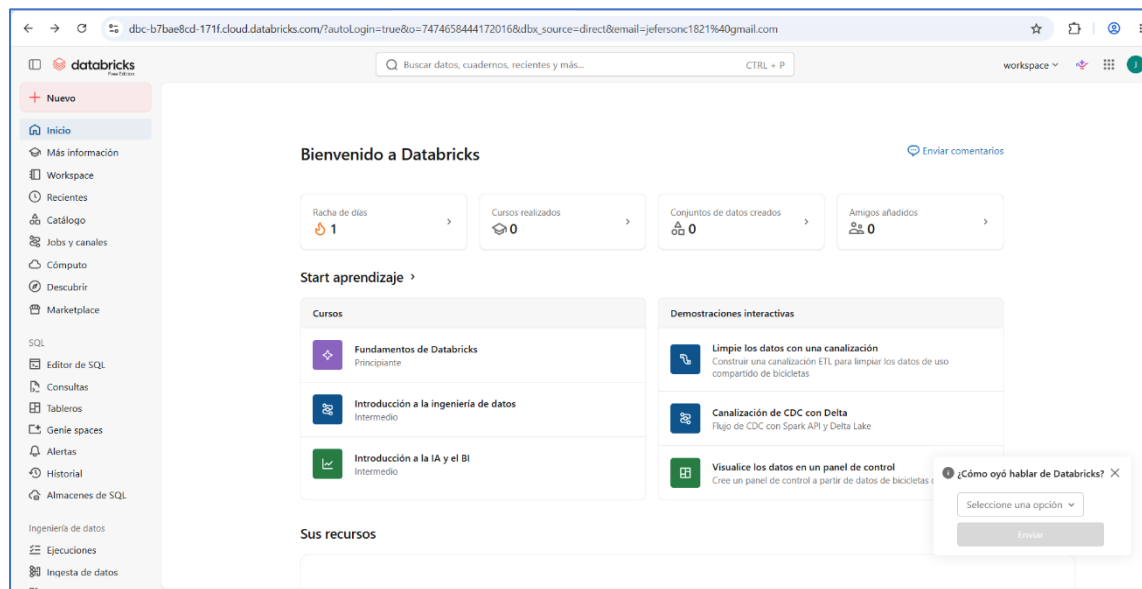
Where are you located?

[Continue](#)

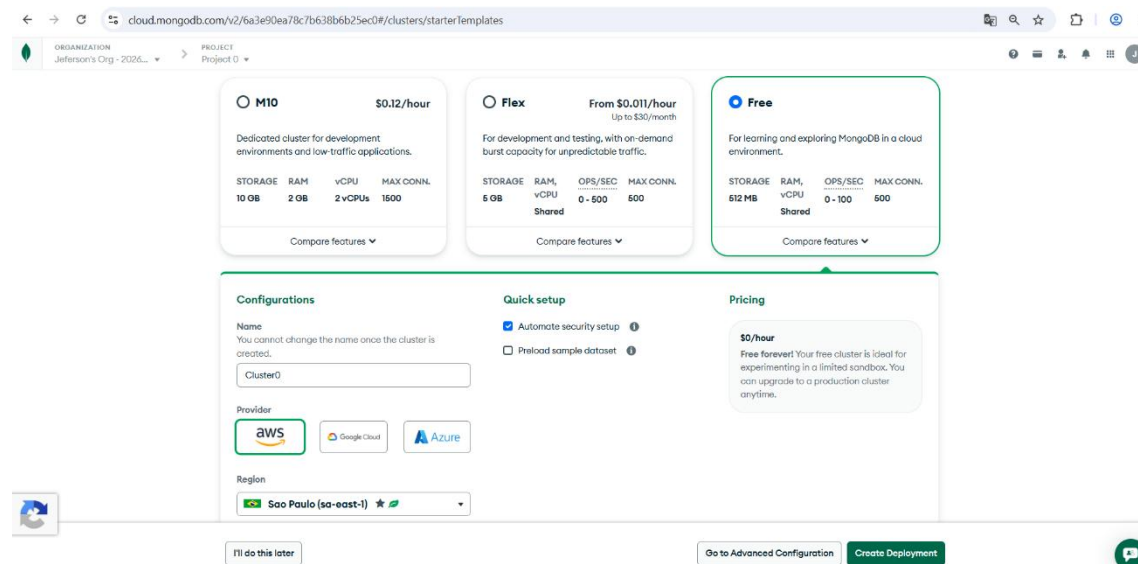
Signing up as jefersonc1821@gmail.com

[Go back](#)

Cuenta creada correctamente:



Creación Cuenta MongoDB/Atlas:



Configuración Driver a Python:

Overview: Automate Security - trueconnectcluster - Cluster0

3

Set up connection security Choose a connection method Connect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver: Python Version: 4.7 or later

2. Install your driver

Run the following on the command line

Note: Use appropriate Python 3 executable

```
python -m pip install "pymongo[srv]"
```

[View MongoDB Python Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ☒ Show Password ☐ View full code sample

```
mongodb+srv://jefersonc1821_db_user:LPmxlfqi1whx2YMJ@cluster0.jp3tdzg.mongodb.net/?appName=Cluster0
```

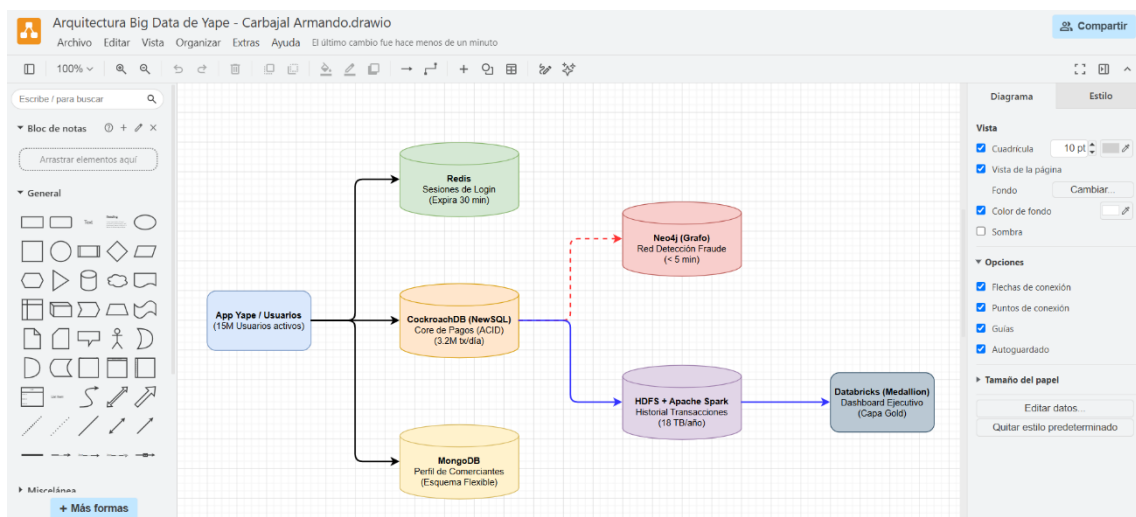
The password for **jefersonc1821_db_user** is included in the connection string for your first time setup. This password will not be available again after exiting this connect flow.

RESOURCES

[Get started with the Python Driver](#) [Python Starter Sample App](#)

[Access your Database Users](#) [Troubleshoot Connections](#)

Creación de Arquitectura Yape en Draw.io:



Creación de Cluster en DataBricks:

Ejecución de Celda 1 Screenshot:

```
dbc-b7bae8cd-171f.cloud.databricks.com/editor/notebooks/364964665401182?o=7474658444172016#command/8447630040213714
```

Buscar datos, cuadernos, recientes y más... CTRL + P

workspace

Nuevo

Inicio

Más información

Workspace

Recientes

Catálogo

Jobs y canales

Cómputo

Descubrir

Marketplace

SQL

Editor de SQL

Consultas

Tableros

Genie spaces

Alertas

Historial

Almacenes de SQL

Ingeniería de datos

Ejecuciones

Ingesta de datos

P2_databricks_yape

Archivo Editar Ver Ejecutar Ayuda Python Última edición ahora

Ejecutar todo Serverless Programar Compartir

Ahora mismo (2 s)

```
1  
"distrito_origen": np.random.choice(distritos, n).tolist(),  
"estado": np.random.choice(estados, n, p=[0.75, 0.1, 0.05, 0.07, 0.03]).tolist(),  
"id_usuario": [f"USR{np.random.randint(1000,9999)}" for _ in range(n)],  
"es_comercio": np.random.choice([True, False], n, p=[0.4, 0.6]).tolist()  
}  
  
df_pandas = pd.DataFrame(data)  
df_bronze = spark.createDataFrame(df_pandas)  
  
# Mantenemos el DF en memoria para las siguientes celdas  
print(f"Bronze layer: {df_bronze.count()} transacciones cargadas en memoria")  
df_bronze.show(5)  
  
> Consultar el rendimiento (2) ✓ Comprobado para mejoras  
> df_bronze: pyspark.sql.connect.dataframe.DataFrame = [id_transaccion: string, fecha: string ... 7 campos adicionales]  
> df_pandas: pandas.core.frame.DataFrame = [id_transaccion: object, fecha: object ... 7 campos adicionales]
```

✓ Bronze layer: 2000 transacciones cargadas en memoria

id_transaccion	fecha	hora	monto_soles	tipo	distrito_origen	estado	id_usuario	es_comercio
YP0000001	2025-01-01	06:25	35.47	persona_a_persona	Surco	completada	USR9055	false
YP0000002	2025-01-01	19:07	4.16	persona_a_comercio	Surco	completada	USR4216	true
YP0000003	2025-01-01	14:35	42.01	retiro_bcp	San Isidro	completada	USR6688	false
YP0000004	2025-01-01	10:47	7.4	retiro_bcp	Los Olivos	completada	USR5364	false
YP0000005	2025-01-01	07:29	1.41	recarga	San Isidro	completada	USR7042	false

only showing top 5 rows

Ejecución de Celda 2 Screenshot:

```
P2_databricks_yape
```

Archivo Editar Ver Ejecutar Ayuda Python Última edición ahora

Ejecutar todo Serverless Programar

Ahora mismo (4 s)

```
3  
.withColumn("categoria_monto",  
    F.when(F.col("monto_soles") < 20, "micro")  
    .when(F.col("monto_soles") < 100, "medio")  
    .otherwise("alto")) \br/>    .withColumn("es_hora_pico",  
    F.when(F.col("hora").between("12:00", "14:00"), True)  
    .when(F.col("hora").between("18:00", "22:00"), True)  
    .otherwise(False)) \br/>    .withColumn("comision_yape",  
    F.when(F.col("tipo") == "persona_a_comercio",  
        F.round(F.col("monto_soles") * 0.015, 2))  
    .otherwise(0.0))  
  
print(f"Silver layer: {df_silver.count()} transacciones válidas")  
print(f"Eliminadas: {df_bronze.count() - df_silver.count()} (rechazadas/pendientes/monto cero)")  
df_silver.groupBy("categoria_monto").count().show()  
  
> Consultar el rendimiento (4) ✓ Comprobado para mejoras  
> df_silver: pyspark.sql.connect.dataframe.DataFrame = [id_transaccion: string, fecha: string ... 10 campos adicionales]
```

✓ Silver layer: 1802 transacciones válidas
Eliminadas: 198 (rechazadas/pendientes/monto cero)

categoria_monto	count
alto	202
micro	634
medio	966

Ejecución Celda 3:

The screenshot displays the Databricks workspace interface. The notebook is titled "P2_databricks_yape" and is in the "Python" language. The code defines two DataFrames: `gold\_comisiones` and `gold\_distritos`. The output shows two tables:

distrito_origen	total_transacciones	volumen_total_soles	ticket_promedio	transacciones_comercio
Independencia	204	8968.04	43.96	75
San Isidro	197	8194.34	41.6	71
Los Olivos	190	8505.19	44.76	79
Callao	186	8929.61	48.01	77
Miraflores	179	8295.37	46.34	65

INGRESOS YAPE POR HORA (comisión comercios):

hora_dia	num_transacciones	ingresos_yape_soles
01	30	21.56
04	17	20.75
03	24	19.47
22	23	18.65
11	25	16.47

only showing top 5 rows

Sentencia	Iniciada el	Tareas	Duración	Filas leídas	Bytes leídos	Reseña de Genie
L38 > gold_comisione	26 jun 2026, 11:23	1/1 completadas	469 ms	0	0 B	✓
L36 > gold_distritos...	26 jun 2026, 11:23	9/9 completadas	637 ms	0	0 B	✓
L6 > df_silver.create...	26 jun 2026, 11:23	0/0 completadas	188 ms	0	0 B	✓

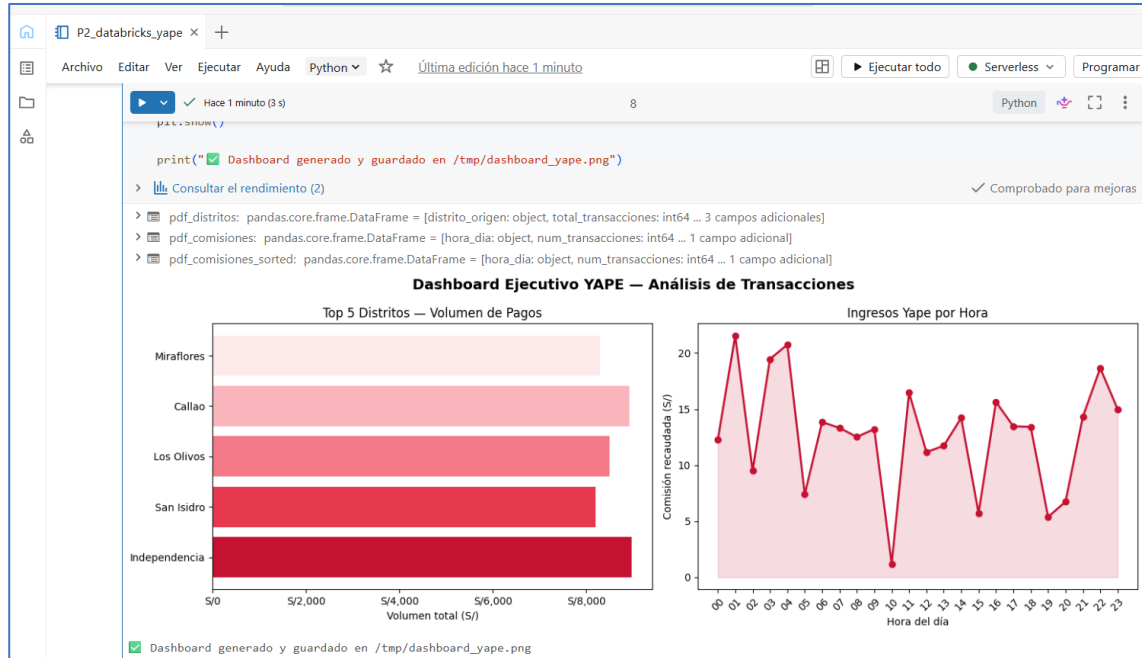
```

gold_comisiones: pyspark.sql.connect.dataframe.DataFrame
  hora_dia: string
  num_transacciones: long
  ingresos_yape_soles: double

gold_distritos: pyspark.sql.connect.dataframe.DataFrame
  distrito_origen: string
  total_transacciones: long
  volumen_total_soles: double
  ticket_promedio: double
  transacciones_comercio: long
+-----+-----+-----+-----+-----+
|distrito origen|total transacciones|volumen total soles|ticket promedio|transacciones comercio|

```

Ejecución Celda 4:



Paso C — MongoDB ATLAS Y Colab:

Instalación:

```
INSTALACIÓN (en Google Colab:)
```

```
[1] pip install pymongo dnspython -q
```

```
... 1.8/1.8 MB 29.1 MB/s eta 0:00:00
```

```
... 331.1/331.1 kB 22.4 MB/s eta 0:00:00
```

Conectado a bd desde COLAB con mi cadena de conexión creado en mongoDB:

```
PASO 1 — Conectar a Atlas
```

```
[2] from pymongo import MongoClient
```

```
[2] import json
```

```
# REEMPLAZA con tu connection string de Atlas:
```

```
CONNECTION_STRING = "mongodb+srv://jefersonc1821_db_user:LPMx1fq1whx2Y9D@cluster0.jp3tdzg.mongodb.net/?appName=Cluster0"
```

```
client = MongoClient(CONNECTION_STRING)
```

```
db = client["yape_db"]
```

```
comerciantes = db["comerciantes"]
```

```
print("✅ Conectado a MongoDB Atlas")
```

```
print(f"DB: {db.name} | Colección: {comerciantes.name}")
```

```
... ✅ Conectado a MongoDB Atlas
```

```
... DB: yape_db | Colección: comerciantes
```

Insertando 5 comerciantes:

```

[3] 1s
{
  "ruc": "20987654321",
  "nombre_comercio": "Distribuidora Norte SAC",
  "tipo": "empresa",
  "representante_legal": "Patricia Luna Torres",
  "distrito": "Independencia",
  "departamento": "Lima",
  "calificacion": 3.9,
  "yape_activo": True,
  "monto_mensual_soles": 85000.00,
  "num_empleados": 45,
  "sectores": ["abarrotes", "limpieza", "bebidas"],
  "clientes_mayoristas": 230,
  "horario": {"apertura": "08:00", "cierre": "18:00"},
  "acepta_delivery": True,
  "zonas_despacho": ["Lima Norte", "Lima Centro"]
}

resultado = comerciantes.insert_many(lista_comerciantes)

print(f"✓ (len(resultado.inserted_ids)) comerciantes insertados en Atlas")
for i, id_in enumerate(resultado.inserted_ids):
    print(f"  {lista_comerciantes[i]['tipo'].upper(): (id_)}")

... 5 comerciantes insertados en Atlas
BODEGA: 6a3eaf3f774348de1b02d832
RESTAURANTE: 6a3eaf3f774348de1b02d833
FARMACIA: 6a3eaf3f774348de1b02d834
TAXI: 6a3eaf3f774348de1b02d835
EMPRESA: 6a3eaf3f774348de1b02d836

```

Paso 3 , Querys con Filtros:

```

[4] 0s
{ 'nombre_comercio': 1, 'monto_mensual_soles': 1, 'distrito': 1, '_id': 0 }
for c in alto_valor:
    print(f" → {c['nombre_comercio']} ({c['distrito']}) S/{c['monto_mensual_soles']:.0f}/mes")
print()

print("CONSULTA 3: Bodegas O farmacias (operador $in)")
bodegas_farmacias = list(comerciantes.find(
    {"tipo": {"$in": ["bodega", "farmacia"]},
    {"nombre_comercio": 1, "tipo": 1, "_id": 0}
))
for c in bodegas_farmacias:
    print(f" → [{c['tipo']}] {c['nombre_comercio']}")

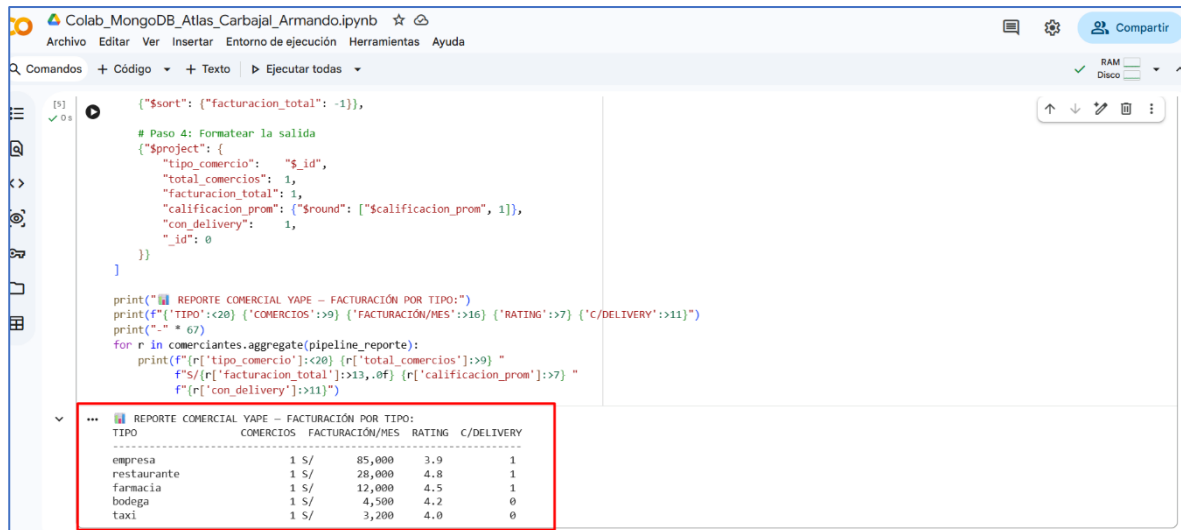
=====
CONSULTA 1: Comerciantes premium (calificación > 4.3 y activos)
★ Cevichería El Muelle SAC (restaurante) - 4.8
★ Farmacia San Pablo Express (farmacia) - 4.5

CONSULTA 2: Comercios con delivery en Lima que facturan > S/10,000/mes
→ Farmacia San Pablo Express (Los Olivos): S/12,000/mes
→ Distribuidora Norte SAC (Independencia): S/85,000/mes

CONSULTA 3: Bodegas O farmacias (operador $in)
→ [bodega] Bodega La Esquina de Don Mario
→ [farmacia] Farmacia San Pablo Express

```


Paso 4 , agregación PIPELINE:



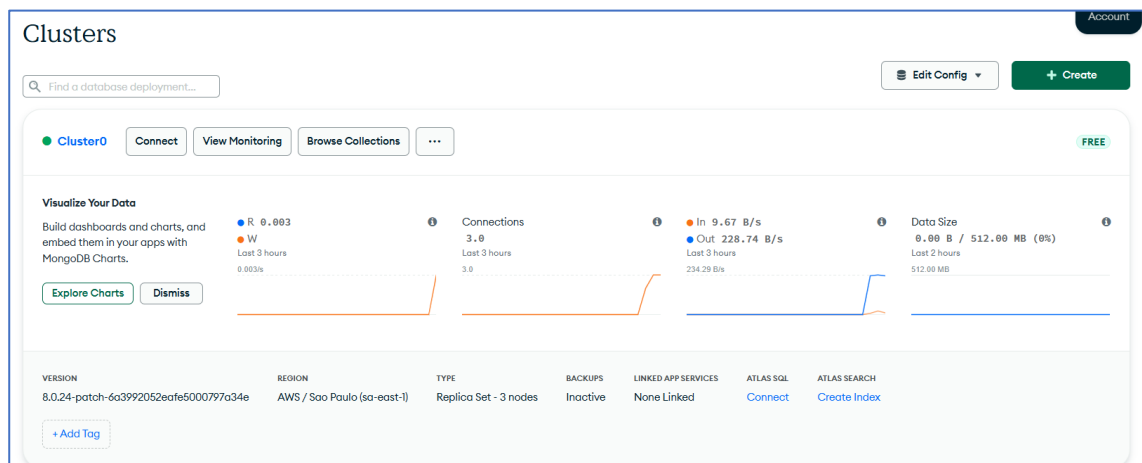
```
{ "$sort": {"facturacion_total": -1}},

# Paso 4: Formatear la salida
{ "$project": {
  "tipo_comercio": "$_id",
  "total_comercios": 1,
  "facturacion_total": 1,
  "calificacion_prom": {"$round": [{"calificacion_prom", 1}],
  "con_delivery": 1,
  "_id": 0
}}

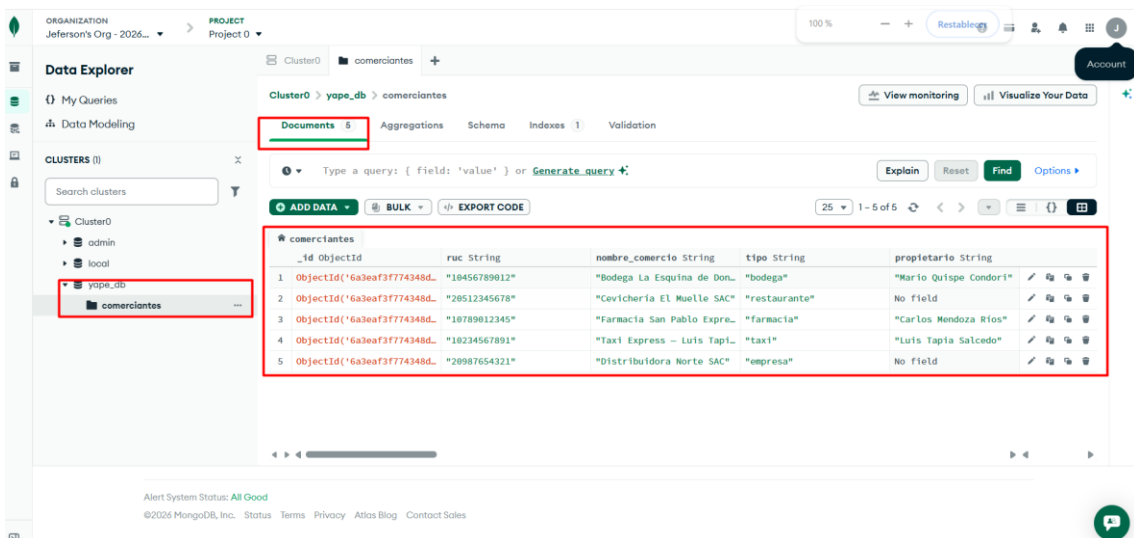
print("\n REPORTE COMERCIAL YAPE - FACTURACIÓN POR TIPO:")
print(f"{'TIPO':<20} {'COMERCIOS':>9} {'FACTURACIÓN/MES':>16} {'RATING':>7} {'C/DELIVERY':>11}")
print("-" * 67)
for r in comerciantes.aggregate(pipeline_reporte):
  print(f"{r['tipo_comercio']:<20} {r['total_comercios']:>9} "
        f"{r['facturacion_total']:>13,.0f} {r['calificacion_prom']:>7} "
        f"{r['con_delivery']:>11}")
```

TIPO	COMERCIOS	FACTURACIÓN/MES	RATING	C/DELIVERY
empresa	1 S/	85,000	3.9	1
restaurante	1 S/	28,000	4.8	1
farmacia	1 S/	12,000	4.5	1
bodega	1 S/	4,500	4.2	0
taxi	1 S/	3,200	4.0	0

ScreenShot de Cluster Mongo DB operando:



Mongo DB ATLAS UI- Collections de 5 Documentos:



Cluster0 > yape_db > comerciantes

Documents 5 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA BULK EXPORT CODE

25 1 - 5 of 5

	_id ObjectId	ruc String	nombre_comercio String	tipo String	propietario String
1	ObjectId('6a3ea3f774348d...')	"18456789012"	"Bodega La Esquina de Don..."	"bodega"	"Mario Quispe Condori"
2	ObjectId('6a3ea3f774348d...')	"28512345678"	"Cevicheria El Muelle SAC"	"restaurante"	No field
3	ObjectId('6a3ea3f774348d...')	"18789012345"	"Farmacia San Pablo Expre..."	"farmacia"	"Carlos Mendoza Rios"
4	ObjectId('6a3ea3f774348d...')	"18234567891"	"Taxi Express - Luis Tapi..."	"taxi"	"Luis Tapia Salcedo"
5	ObjectId('6a3ea3f774348d...')	"28987654321"	"Distribuidora Norte SAC"	"empresa"	No field

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Paso D: DOCKER DESKTOP

Instalación de MongoDB en Docker:

```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Windows\system32> docker pull mongo:7.0
7.0: Pulling from library/mongo
c173d96c3dbf: Pull complete
40d16f30db40: Pull complete
87d5f0fd138b: Pull complete
e7ffdd89d14a: Pull complete
ac9ee532aa31: Pull complete
d55e0d5a1bc8: Pull complete
24325f4d7fe6: Pull complete
19de43bd888b: Pull complete
f63d1a28ef75: Download complete
0f3448d9cfaa: Download complete
Digest: sha256:8ecb514b00bdcc0bde67ef4e6c330385377a9dc68e24ee94e28c07c891647348
Status: Downloaded newer image for mongo:7.0
docker.io/library/mongo:7.0

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview mongo:7.0
PS C:\Windows\system32>
```

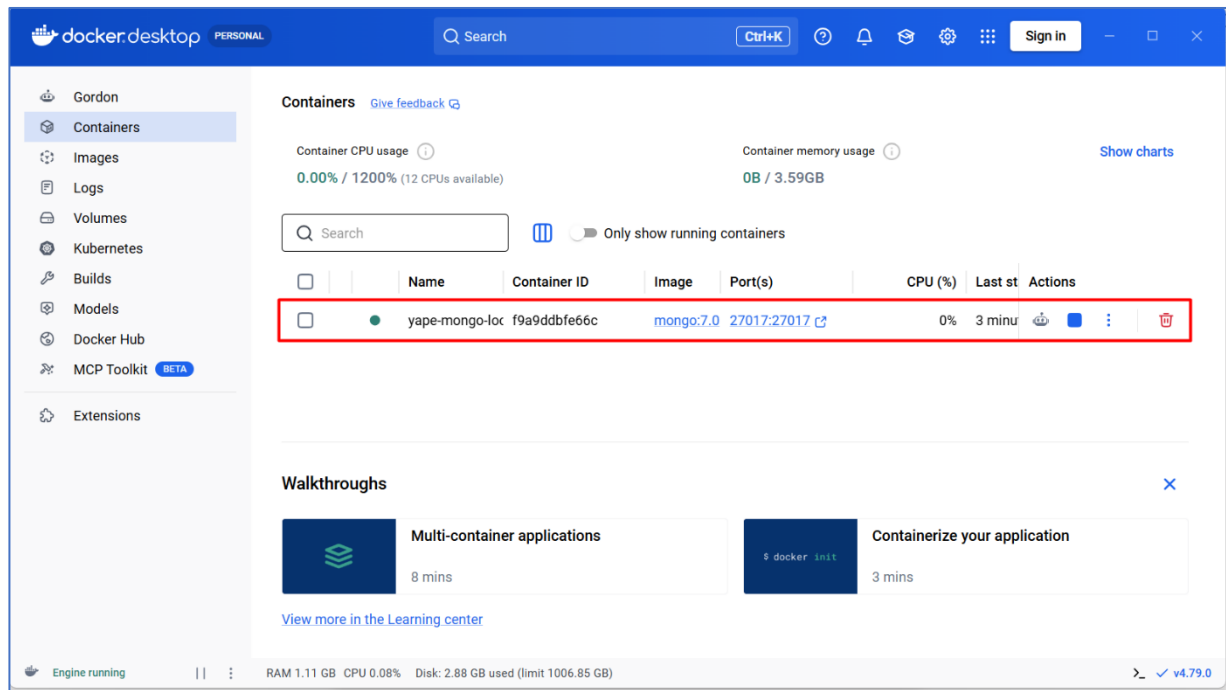
Levantar el contenedor:

```
View a summary of image vulnerabilities and recommendations → docker scout quickview mongo:7.0
PS C:\Windows\system32> docker run -d --name yape-mongo-local -p 27017:27017 -e MONGO_INITDB_ROOT_USERNAME=admin -e MONGO
0_INITDB_ROOT_PASSWORD=yape2026 mongo:7.0
f9a9ddbfe66cab5d1fca640e3ccc961e8afdf9cbe8e79f727162847f5f616f3
PS C:\Windows\system32>
```

Verificar el contenedor:

```
PS C:\Windows\system32> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
f9a9ddbfe66c   mongo:7.0  "docker-entrypoint.s..." 53 seconds ago Up 51 seconds  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
yape-mongo-local
PS C:\Windows\system32>
```

Docker Desktop – Mongo DB Local Online:



Instalando Mongo en PC Local:

```

C:\Windows\System32>pip install pymongo
Collecting pymongo
  Downloading pymongo-4.17.0-cp314-cp314-win_amd64.whl.metadata (10 kB)
Collecting dnspython<3.0.0,>=2.6.1 (from pymongo)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloaded pymongo-4.17.0-cp314-cp314-win_amd64.whl (1.0 MB)
----- 1.0/1.0 MB 3.7 MB/s 0:00:00
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.8.0 pymongo-4.17.0

C:\Windows\System32>

```

Habilitando en Vs Code:

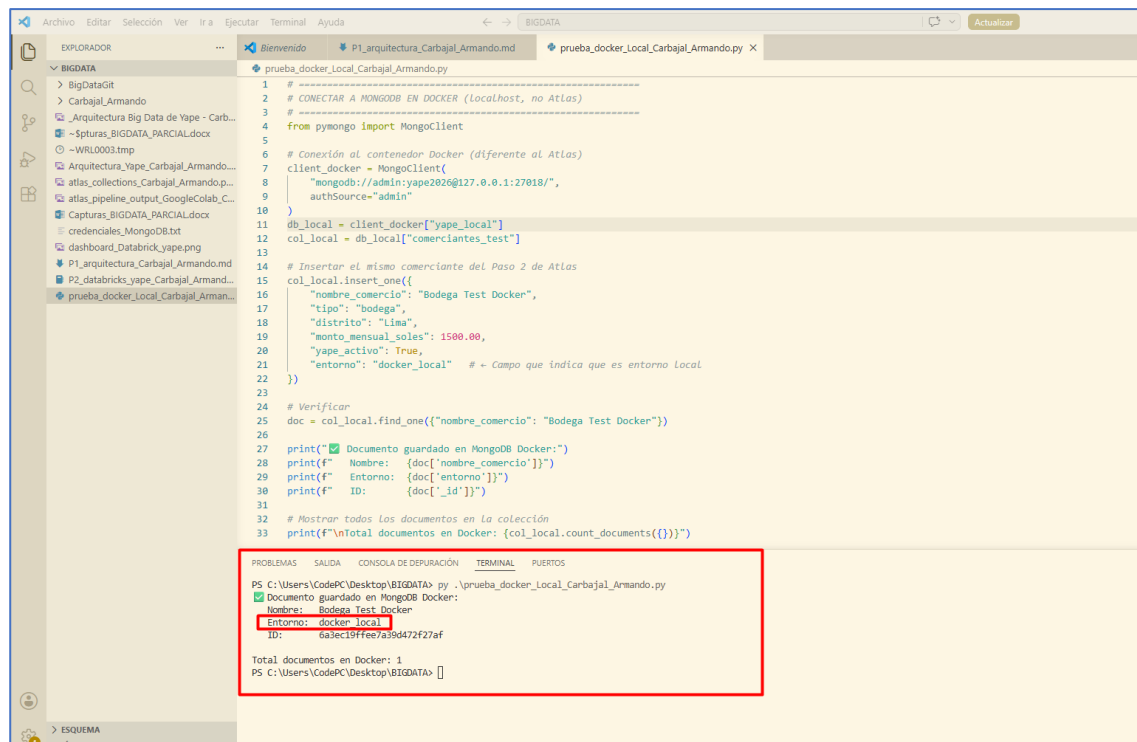
```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

PS C:\Users\CodePC\Desktop\BIGDATA> py -m pip install pymongo
Collecting pymongo
  Using cached pymongo-4.17.0-cp314-cp314-win_amd64.whl.metadata (10 kB)
Collecting dnspython<3.0.0,>=2.6.1 (from pymongo)
  Using cached dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Using cached pymongo-4.17.0-cp314-cp314-win_amd64.whl (1.0 MB)
Using cached dnspython-2.8.0-py3-none-any.whl (331 kB)
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.8.0 pymongo-4.17.0
PS C:\Users\CodePC\Desktop\BIGDATA>

```

Prueba de ejecución Mongo Local:



```
1 # =====
2 # CONECTAR A MONGODB EN DOCKER (localhost, no Atlas)
3 # =====
4 from pymongo import MongoClient
5
6 # Conexión al contenedor Docker (diferente al Atlas)
7 client_docker = MongoClient(
8     "mongodb://admin:yape2026@127.0.0.1:27018/",
9     authSource="admin"
10 )
11 db_local = client_docker["yape_local"]
12 col_local = db_local["comerciantes_test"]
13
14 # Insertar el mismo comerciante del Paso 2 de Atlas
15 col_local.insert_one({
16     "nombre_comercio": "Bodega Test Docker",
17     "tipo": "bodega",
18     "distrito": "Lima",
19     "monto_mensual_soles": 1500.00,
20     "yape_activo": True,
21     "entorno": "docker_local" # + Campo que indica que es entorno Local
22 })
23
24 # Verificar
25 doc = col_local.find_one({"nombre_comercio": "Bodega Test Docker"})
26
27 print("✅ Documento guardado en MongoDB Docker:")
28 print(f"  Nombre: {doc['nombre_comercio']}")
29 print(f"  Entorno: {doc['entorno']}")
30 print(f"  ID: {doc['_id']}")
31
32 # Mostrar todos los documentos en la colección
33 print(f"Total documentos en Docker: {col_local.count_documents({}})")
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
PS C:\Users\CodePC\Desktop\BIGDATA> py .\prueba_docker_Local_Carbajal_Armando.py
✅ Documento guardado en MongoDB Docker:
Nombre: Bodega Test Docker
Entorno: docker_local
ID: 6a3ec19ffef7a350472f27af

Total documentos en Docker: 1
PS C:\Users\CodePC\Desktop\BIGDATA> []
```